



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Case Analysis of Binary Search Algorithm

Best, Average, and Worst-Case Time Complexity Analysis Using Asymptotic Notation

Presented by:

NIVASINI VM (192511356)

CSA0613 :DESIGN AND ANALYSIS OF ALGORITHM

Technical Presentation



INTRODUCTION

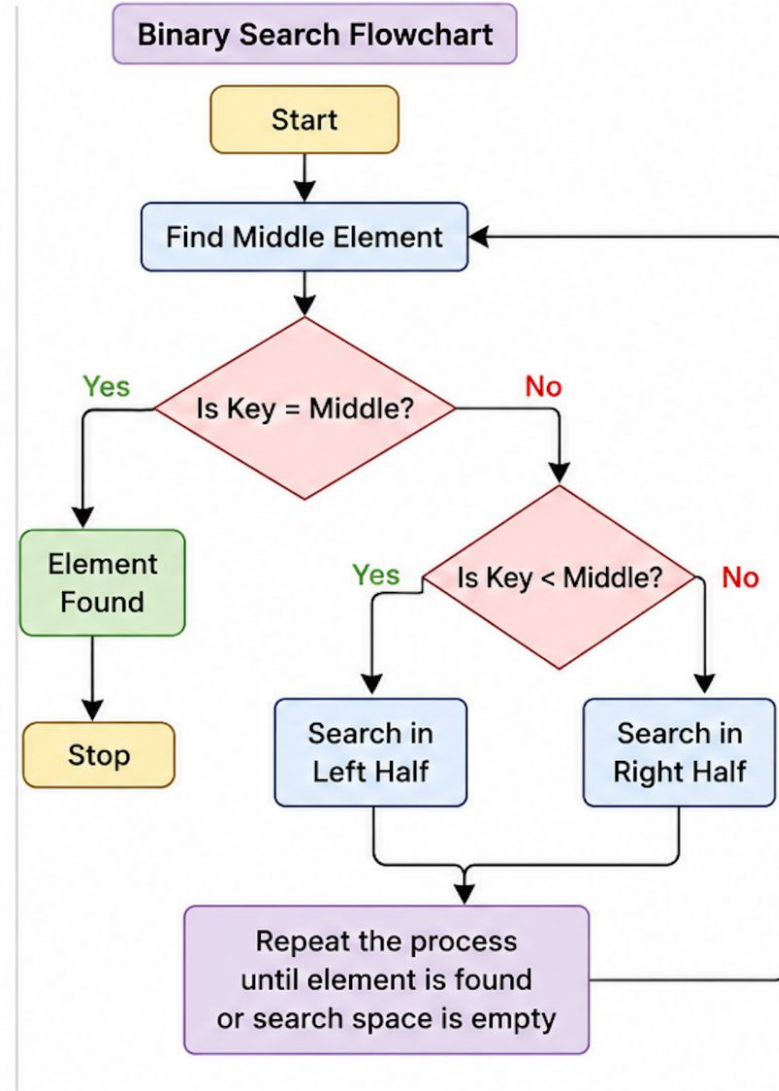
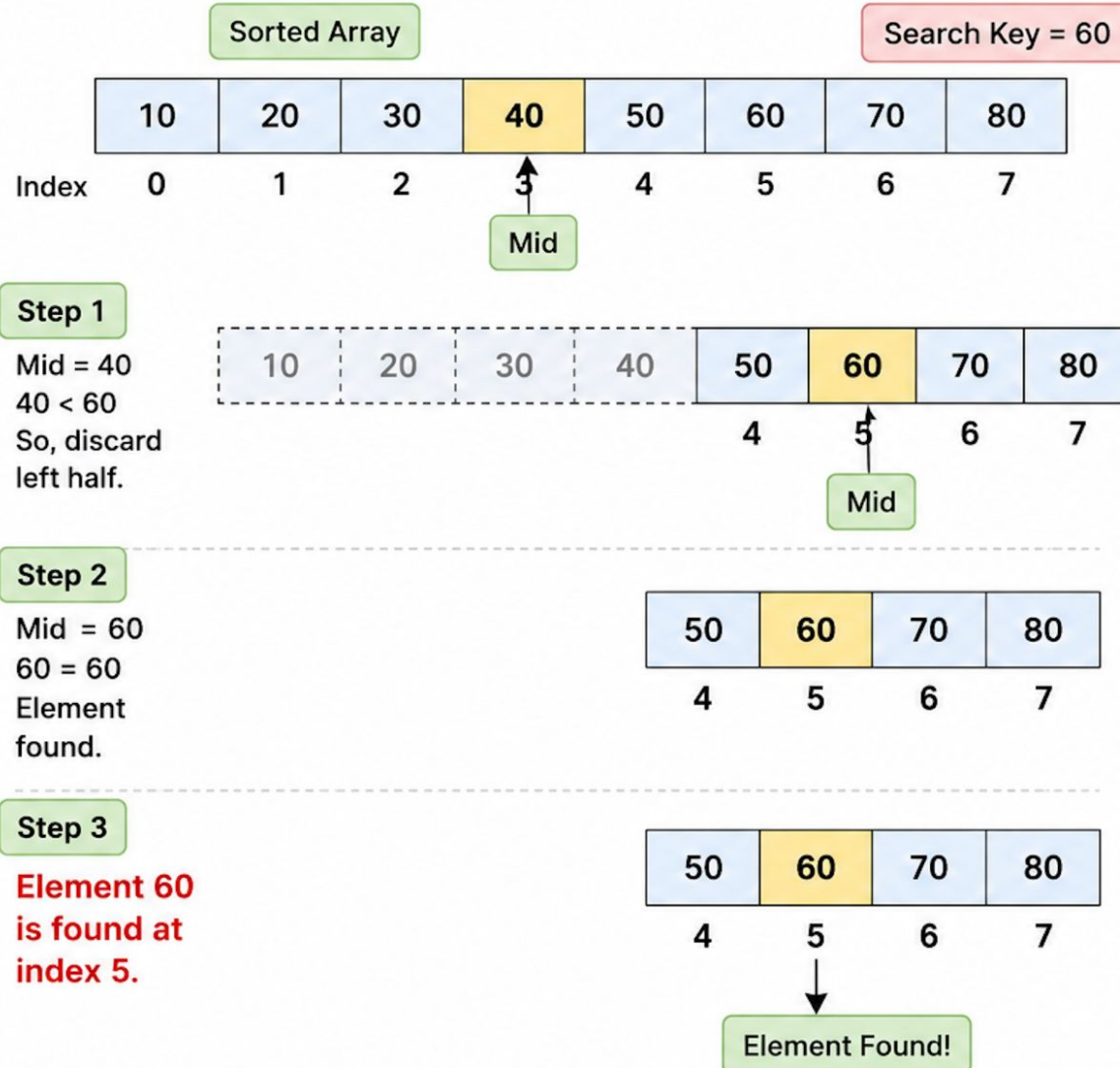


What is Binary Search?

- Binary Search is an efficient searching algorithm.
- Works only on **sorted arrays**.
- Repeatedly divides the search space into two halves.
- Compares the target with the middle element.
- Time complexity is much better than Linear Search for large datasets.

WORKING OF BINARY SEARCH

Binary Search works on a sorted array by repeatedly dividing the search space into two halves.





BEST CASE ANALYSIS



Condition

The target element is exactly at the middle position during the first comparison.

Example

10 20 30 40 50 60 70

Search = 40

40 ✓

Analysis

- Comparisons = 1
- Element found immediately
- Time Complexity:

$O(1)$

Justification

Only one comparison is required regardless of input size.



AVERAGE CASE ANALYSIS



Condition

The element is found after several divisions.

Example

10 20 30 40 50 60 70

Search = 60

40

\

60 ✓

Analysis

- Search space reduces by half every iteration.
- Average comparisons $\approx \log_2 n$

Time Complexity

$O(\log n)$

WORST CASE ANALYSIS

Condition

- Element is located at the last possible position
OR
- Element is not present.

Example

Search = 75

40

\

60

\

70

\

Not Found

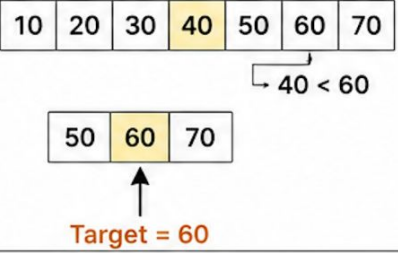
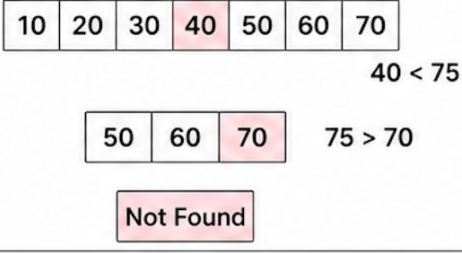
Analysis

Maximum number of comparisons

Time Complexity

$O(\log n)$

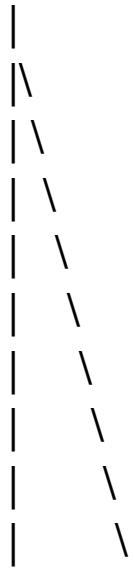
CASE COMPARISON TABLE – BINARY SEARCH

Case	Best Case	Average Case	Worst Case
Condition	The target element is exactly at the middle position during the first comparison.	The target element is found after several comparisons (not in the first step).	The target element is at the last possible position OR the element is not present in the array.
Example			
Comparisons	1	$\approx \log_2 n$	$\leq \log_2 n$
Time Complexity (Asymptotic Notation)	$O(1)$	$O(\log n)$	$O(\log n)$
Number of Steps	Minimum (1 step)	Moderate (Depends on position)	Maximum ($\approx \log_2 n$ steps)
Space Complexity	$O(1)$ (Iterative) $O(\log n)$ (Recursive)	$O(1)$ (Iterative) $O(\log n)$ (Recursive)	$O(1)$ (Iterative) $O(\log n)$ (Recursive)
Explanation	Element found in the first comparison. No further search is needed.	Search space is reduced by half each time on average.	Search continues until the last element or until the search space becomes empty.

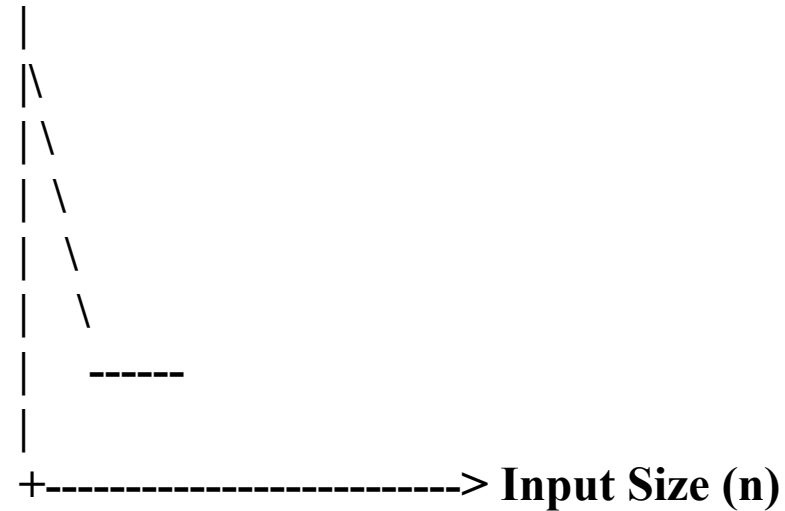
BEHAVIOR DIAGRAM

Number of Comparisons

Linear Search



Binary Search



Interpretation

- Linear Search grows linearly.
- Binary Search grows logarithmically.



PRACTICAL RELEVANCE



Which Case is Most Relevant?

Average Case

- Most practical scenario.
- Usually we do not know the element's position.
- Binary Search performs efficiently for large datasets.

Worst Case

- Important for performance guarantees.
- Used while analyzing algorithm efficiency.

Best Case

- Rare in practical applications.

Most Relevant in Practice: Average Case ($O(\log n)$)



CONCLUSION



- Binary Search requires a **sorted array**.
- It divides the search space into halves repeatedly.
- Best Case: **$O(1)$**
- Average Case: **$O(\log n)$**
- Worst Case: **$O(\log n)$**
- Binary Search is much faster than Linear Search for large datasets.



Thank You!!!

CO1 AT3 TECHNICAL PRESENTATION

NAME:NIVASINI VM

REG NO:192511356

COURSE:CSA 0613 DESIGN AND ANALYSIS OF ALGORITHM



Searching is one of the most common operations in computer science, and choosing an efficient searching algorithm is essential for improving system performance. Binary Search is a powerful and efficient searching algorithm that works only on sorted arrays. Unlike Linear Search, which checks every element one by one, Binary Search repeatedly divides the search space into two equal halves, significantly reducing the number of comparisons required to find the target element. This makes it much faster, especially when working with large datasets.

In this presentation, I have explained the working principle of Binary Search and analyzed its performance in the best, average, and worst cases using asymptotic notation. The time complexities of these cases help us understand how efficiently the algorithm performs under different conditions. Finally, I discussed its practical relevance and why Binary Search is widely used in real-world applications where fast and efficient searching is required.